# ESMM: Exploiting Emergent Sparsity in Matrix Multiplication for Dynamic LLM Inference

Avery Clapp
Johns Hopkins University
Baltimore, Maryland, USA
aclapp1@jhu.edu

## Abstract

Transformer inference is dominated by matrix multiplications where activation matrices exhibit 50–90% sparsity at runtime, yet this sparsity is input-dependent, changing with every forward pass, rendering it inaccessible to existing sparse kernels that require static structure known at load time. The key challenge is not detecting sparsity, but doing so cheaply enough that detection overhead does not erase the computational savings. We observe that block-structured sparsity in both the activation matrix $A$ and weight matrix $B$ enables warp-uniform skip decisions over inner-dimension tile pairs, avoiding divergent execution entirely. ESMM exploits this observation through a CUDA kernel family that extracts joint activation–weight sparsity patterns at runtime and skips zero-contributing tile pairs within a single matrix multiplication, requiring no format conversion, model modification, or retraining. On NVIDIA A10G at $4096 \times 4096$, ESMM achieves 2.65× compute speedup over cuBLAS at 87.5% block sparsity, with breakeven near 50%. Evaluation on real LLaMA weights reveals that standard group-structured pruning produces negligible block sparsity below 99% element sparsity — a structural consequence of group-level pruning constraints highlighting block-aware pruning as the critical open problem for realizing these gains in practice.

**ACM Reference Format:**
Avery Clapp. 2026. ESMM: Exploiting Emergent Sparsity in Matrix Multiplication for Dynamic LLM Inference. In *Proceedings of Conference Name (Workshop)*. ACM, New York, NY, USA, 6 pages.

## 1 Introduction

Matrix multiplications in transformer feed-forward networks account for over 60% of LLM inference latency [2]. Recent work has shown that ReLU and SiLU activations produce 50–90% zero-valued intermediate activations [5, 6] — a phenomenon termed emergent activation sparsity. Unlike weight sparsity, activation sparsity is dynamic: patterns are input-dependent and unknown until the forward pass, making them impossible to preprocess offline. If exploitable at runtime, this sparsity represents a zero-cost opportunity: the same model, the same weights, but faster inference with no accuracy tradeoff or retraining.

Exploiting dynamic sparsity on GPUs is fundamentally difficult. Warp-based execution requires all 32 threads to follow a uniform control path. Element-level sparsity scattered across a matrix produces divergence, serializing computation instead of skipping it. Meaningful speedup requires block-structured sparsity, where entire aligned regions are zero, enabling warp-uniform skip decisions. Even given sufficient block structure, sparsity patterns must be constructed at runtime cheaply enough that detection overhead does not erase the computational savings.

Prior approaches do not solve this problem. SpInfer [2] and Flash-LLM [10] achieve strong results for static weight sparsity by preprocessing matrices offline into sparse formats — an approach inapplicable when patterns change every forward pass. SpInfer explicitly scopes itself to static sparsity and does not support dynamic activation patterns [2]. Methods targeting activation sparsity — Deja Vu [6] and PIT [1] — require either a trained predictor module or model retraining, limiting applicability to standard deployed models. Existing approaches therefore either require offline preprocessing, model retraining, or prediction modules, leaving a fundamental question: can dynamic sparsity be exploited inline with minimal runtime overhead?

Unlike prior approaches, ESMM exploits both runtime activation sparsity and static weight sparsity jointly, without retraining or offline format conversion. We present **ESMM**, a fused CUDA kernel that detects block-structured sparsity inline during matrix multiplication and skips inactive K-blocks within a single kernel launch. Weight matrix sparsity patterns are extracted once offline and amortized across tokens; activation patterns are detected at runtime via warp ballot primitives. On NVIDIA A10G at $4096 \times 4096$, ESMM achieves up to 2.65× compute speedup (2.32× end-to-end including preprocessing) over cuBLAS at 87.5% block sparsity, with breakeven near 50% block sparsity.

**Summary of Contributions:**

- The discovery that joint block-structured sparsity in activation and weight matrices enables warp-uniform skip decisions, allowing dynamic activation sparsity to be exploited efficiently on GPUs.
- ESMM, a fused CUDA kernel that detects activation sparsity inline and skips zero-contributing K-blocks within a single kernel launch, requiring no format conversion or model modification (Section 2).
- Empirical evaluation demonstrating up to 2.65× compute speedup (2.32× end-to-end) over cuBLAS at 87.5% block sparsity on NVIDIA A10G (Section 3).
- A characterization of the granularity–overhead tradeoff for runtime sparsity detection, identifying $32 \times 8$ tiles and ∼50% block sparsity as the practical breakeven point (Section 3).

Our results demonstrate that runtime sparsity, if efficiently exploited, can provide substantial speedups for LLM inference without altering the model, opening a new avenue for high-throughput GPU inference.

*Workshop, City, Country*
2026.

## 2 ESMM Design

ESMM introduces joint runtime detection of activation and weight sparsity, enabling skip decisions at block, warp, and dotIdx levels without format conversion or retraining. We refer to our primary kernel as ESMM; ablation variants are described in Section 3.

### 2.1 Joint Sparsity Condition

ESMM targets matrix multiplications of the form $C = AB$ where $A \in \mathbb{R}^{M \times K}$ and $B \in \mathbb{R}^{K \times N}$. We partition the K-dimension into K-tiles of width $BK = 8$, and partition $A$ into row-groups of height $BM = 32$ (one per warp) and $B$ into column-groups of width $BN = 32$. A K-tile at index $k$ contributes to output K-tile $(warpRow, warpCol)$ only if the corresponding A row-group and B column-group are both nonzero along that K-tile. Formally, the skip condition is:

$$\text{joint}[r][c][k] = \texttt{a_pat}[r][k] \ \& \ \texttt{b_pat}[c][k] \tag{1}$$

where $r = warpRow$, $c = warpCol$, and & is bitwise AND over the 8-bit column masks. If $\text{joint}[r][c][k] = 0$, the entire outer product contribution of that K-tile to the corresponding output region is zero and can be skipped without loss of correctness.

The joint condition is strictly more powerful than single-matrix sparsity. With A-only sparsity, a K-tile can be skipped only when the entire A row-group is zero — a condition that becomes increasingly rare as A density grows. With the joint condition, a K-tile is skippable whenever *either* matrix is zero along that K-tile for that warp pair. At 25% block density, this produces 59.8% warp-level skip rates and 6.0% block-level skip rates, compared to near-zero block skips under single-matrix sparsity at the same density.

This condition naturally induces a three-level hierarchy of skip opportunities, exploiting coarser structure first and falling through to finer checks only when necessary. Each level is zero-overhead in the common case. Details are described in Section 2.3. Figure 1 illustrates the hierarchy.

(1) **Block-level**: all warps in a CUDA block skip K-tile $k$ simultaneously
(2) **Warp-level**: an individual warp skips K-tile $k$ while others compute
(3) **dotIdx-level**: a single K-column within an otherwise-active K-tile is skipped

**Figure 1: Three-level skip hierarchy in ESMM. Block-level skips eliminate entire K-tiles for all warps; warp-level skips allow individual warps to bypass inactive K-tiles; dotIdx-level skips handle residual sparsity within active K-tiles.**

### 2.2 Pattern Representation and Preprocessing

ESMM encodes sparsity as compact uint8 bitmasks, one per $(warpRow, k)$ pair for $A$ and one per $(warpCol, k)$ pair for $B$. Each bit $d$ of the mask indicates whether K-column $k \cdot BK + d$ contains any nonzero value within that row- or column-group. This representation is dense in memory, requires no pointer indirection, and supports the joint condition via a single bitwise AND. Figure 2 shows the mapping of $32 \times 8$ K-tiles to uint8 patterns.

**Figure 2: K-tile layout and bitmask encoding. Each $32 \times 8$ K-tile of $A$ and $B$ maps to a single uint8 pattern. Joint patterns are preloaded into shared memory before the K-loop begins.**

Two lightweight preprocessing kernels extract these patterns before each GEMM:

- `preprocess_a` scans $A$ in $32 \times 8$ K-tiles using vectorized float2 loads, producing `a_patterns[numMBlocks][numKBlocks]`.
- `preprocess_b` scans $B$ in $32 \times 8$ K-tiles using warp-level primitives to aggregate nonzero flags across columns, producing `b_patterns[numNBlocks][numKBlocks]`.

Combined preprocessing cost is ~375 $\mu$s for $4096 \times 4096$, independent of sparsity level — the kernels scan all elements regardless of their values. For weight matrices, this cost is paid once at load time and amortized across all tokens in a batch. For activation matrices, it is paid once per forward pass.

Before the main K-loop begins, each CUDA block preloads all joint patterns for its assigned warp pairs into shared memory. For warp $w$ and K-tile $k$:

$$\texttt{js}[w][k] = \texttt{a_pat}[\text{warpRow}(w)][k] \ \& \ \texttt{b_pat}[\text{warpCol}(w)][k] \tag{2}$$

where `js` denotes the joint shared memory array, $w \in [0, 8)$, and $k \in [0, \text{numKBlocks})$. This preload is a one-time cost per block. All subsequent pattern lookups in the K-loop read from shared memory rather than global memory, eliminating repeated gmem accesses in the hot path. Shared memory for `js` is sized at compile time as `NUM_WARPS` × `MAX_K_BLOCKS` and template-dispatched at 128, 256, 512, or 1024 based on the actual K dimension. Template dispatch preserves occupancy, ensuring all warps remain fully utilized across different matrix sizes.

### 2.3 Skip Logic

Algorithm 1 shows the three-level skip logic in the K-loop.

**Algorithm 1** ESMM K-loop with three-level skipping

```
1:  for k = 0 to numKBlocks − 1 do
2:      if block_joint[k] = 0 then                ▷ Level 1: block skip
3:          continue                              ▷ No sync, no memory access
4:      end if
5:      Load A K-tile (float2), load B K-tile (float4) into smem
6:      __syncthreads()
7:      if js[warpId][k] = 0 then                 ▷ Level 2: warp skip
8:          continue
9:      end if
10:     for d = 0 to BK − 1 do                    ▷ Level 3: dotIdx
11:         if bit d of js[warpId][k] is set then
12:             Accumulate outer product for column d
13:         end if
14:     end for
15:     __syncthreads()
16: end for
```

**Block-level skip** (line 2) checks $\texttt{block_joint}[k] = \bigvee_w \texttt{js}[w][k]$, precomputed during the smem preload phase. When zero, all 256 threads execute `continue` simultaneously with no divergence and no shared memory access. This avoids both K-tile loads and the synchronization barriers that would otherwise be required. This is the cheapest possible skip.

**Warp-level skip** (line 7) allows individual warps to bypass the dotIdx loop even when other warps in the block are active. Each warp reads its own joint byte from shared memory. If zero, it skips all eight inner iterations. At 12.5% density, 88.1% of warp-K-tile pairs are skipped at this level, reducing executed dotIdx operations to just 1.6% of the dense baseline.

**dotIdx-level skip** (line 10) handles residual sparsity within otherwise active K-tiles. The inner loop iterates over $BK = 8$ bits with `#pragma unroll`, checking each bit of the joint byte. Only set bits trigger an outer product accumulation.

## 2.4 Tile Sizing and Memory Design

**K-tile width.** $BK = 8$ is chosen so that a single uint8 encodes exactly one K-tile's worth of columns. It also aligns with float4 B K-tile loading (4 floats × 2 = 8 elements per row). Coarser K-tiles reduce pattern overhead, but they miss zeros within a K-tile. Finer K-tiles increase metadata volume and degrade memory efficiency.

**Warp-row granularity.** $BM = 32$ matches the natural warp width, making each warp independently responsible for a 32-row stripe of $A$ — the minimum granularity for independent warp-level skipping. An earlier design using $BM = 64$ warp-row groups produced roughly half the skip opportunities at moderate sparsity, as a 64-row group is less likely to be entirely zero than a 32-row group.

**A K-tile load width.** ESMM uses float2 loads for the A K-tile, achieving 100% thread utilization during K-tile loading (rowStride = $BM = 32$, one thread per row). A float4 alternative halves load instruction count but leaves half the threads idle (rowStride = $2 \times BM = 64$). Float2 is the better choice at low sparsity levels where A K-tile load time is non-negligible relative to near-zero compute. ESMM uses float2 to optimize for the sparse regime.

**Overhead at full density.** ESMM runs at 12.10 ms versus cuBLAS at 7.19 ms at 100% density — a 0.59× slowdown. This overhead comes from preprocessing (~375 $\mu$s), smem preload of joint K-tile patterns, and per-K-tile skip checks. Break-even occurs near 50% block sparsity, above which skip savings outweigh overhead.

**Generalization.** ESMM targets Ampere-class GPUs (sm_80) and is expected to generalize to other Ampere and Hopper devices with similar warp execution models. Performance characteristics will vary with memory bandwidth and L2 cache capacity, but the skip hierarchy and bitmask representation are not architecture-specific.

# 3 Evaluation

## 3.1 Experimental Setup

All experiments run on an NVIDIA A10G GPU (Ampere, sm_80, 24 GB) on AWS EC2. Kernels are compiled with `nvcc -O3 -use_fast_math -arch=sm_80`. Compute times are measured with NVIDIA Nsight Compute (NCU), excluding preprocessing overhead; end-to-end times use CUDA events averaged over 10 runs. The 4096 × 4096 configuration reflects the dominant projection sizes in LLaMA-class transformer layers, making it representative of practical inference workloads. The cuBLAS baseline uses `cublasSgemm` with `CUBLAS_OP_N` in row-major layout via a transposed call.[1]

Synthetic experiments use 4096 × 4096 matrices with blockwise sparsity: each $BM = 32$ row block × $BK = 8$ K-tile is independently zeroed with probability (1 − density). This directly instantiates the structure ESMM targets and therefore represents an idealized upper bound on achievable speedup. Real-weight experiments use Wanda [9] and SparseGPT [3] pruned LLaMA-7B Layer 0 weight tensors loaded from disk, paired with a dense random activation matrix $A$. Row and column permutations are applied to $B$ prior to evaluation to maximize block sparsity.

## 3.2 Main Result: Speedup vs. Sparsity

Table 1 reports ESMM compute and end-to-end times against cuBLAS across synthetic blockwise densities. Figure 3 plots both speedup curves as a function of block density.

**Table 1: ESMM vs. cuBLAS on synthetic blockwise sparsity, 4096 × 4096. Compute times from NCU; end-to-end adds ~374 $\mu$s preprocessing. Bold marks peak speedup at 12.5% density.**

| | Block Density | | | |
|---|---|---|---|---|
| | 100% | 50% | 25% | 12.5% |
| cuBLAS | 7.19 ms | 7.19 ms | 7.19 ms | 7.19 ms |
| ESMM compute | 12.10 ms | 6.12 ms | 4.04 ms | 2.72 ms |
| ESMM end-to-end | 12.47 ms | 6.49 ms | 4.41 ms | 3.09 ms |
| Compute speedup | 0.59× | 1.17× | 1.78× | **2.65×** |
| End-to-end speedup | 0.58× | 1.11× | 1.63× | **2.32×** |

At 12.5% block density, ESMM achieves 2.65× **compute speedup** and 2.32× **end-to-end speedup** over cuBLAS. At 25% density the gains are 1.78× compute and 1.63× end-to-end. These results directly validate the three-level skip hierarchy and shared memory

[1] Row-major SGEMM is implemented as a transposed column-major call to match cuBLAS conventions.

bitmask caching described in Section 2: block-level skips eliminate entire K-tiles before any memory is touched, warp-level skips handle the remainder, and smem-cached patterns avoid repeated global memory reads in the hot loop.

**Break-even occurs near 50% block density**, below which ESMM consistently outperforms cuBLAS. Above this threshold, preprocessing and shared-memory overhead outweigh skip savings. At 100% density ESMM runs at 0.59× cuBLAS, a slowdown that confirms the cost of pattern management at full density. Preprocessing is ∼374 $\mu$s flat across all sparsity levels; for weight matrices reused across a token batch this amortizes to near zero, while for activation matrices it is paid once per forward pass.

**Figure 3: ESMM compute and end-to-end speedup over cuBLAS vs. block density. Dashed line at** 1.0× **marks cuBLAS parity. Break-even near 50% density;** 2.65× **compute speedup at 12.5%.**

## 3.3 Ablation: Design Decisions

Table 2 traces the kernel evolution from the initial design (K20) to the final configuration. Figure 4 visualizes compute times across all variants. Three design decisions account for the cumulative improvement at 12.5% density.

**Table 2: Kernel ablation, NCU compute times (ms),** 4096×4096**, synthetic blockwise sparsity. K27 isolates 32-row granularity without block-level skipping, revealing that finer granularity alone is insufficient. "—" indicates the configuration was not measured.**

| Kernel | Description | 100% | 50% | 25% | 12.5% |
|---|---|---|---|---|---|
| cuBLAS | Dense baseline | 7.19 | 7.19 | 7.19 | 7.19 |
| K20 | 64-row, float4 A | 11.65 | 6.32 | 4.16 | 3.80 |
| K27 | 32-row, no block skip | — | 6.01 | 4.51 | 4.41 |
| K26 | 32-row + block skip | 12.94 | 6.02 | 4.22 | 2.85 |
| K28 | gmem patterns, 32-row | 13.08 | 6.20 | 4.22 | 3.34 |
| ESMM | smem patterns + float2 A | 12.10 | 6.12 | 4.04 | **2.72** |

**32-row granularity + block-level skip (K26 vs. K27):** −35% **at 12.5% density.** Halving warp-row groups from 64 to 32 rows alone (K27) does not improve performance — K27 runs at 4.41 ms at 12.5% density, worse than K20's 3.80 ms. Finer granularity creates more potentially skippable tiles, but without block-level skipping there is no mechanism to exploit them. Adding block-level skipping (K26) drops runtime to 2.85 ms. Finer granularity is necessary but not sufficient; block-level skipping converts tile structure into saved work.

**Shared memory pattern caching (ESMM vs. K28):** −19% **at 12.5% density.** K28 reads joint patterns from global memory on every K-tile iteration. ESMM preloads all joint patterns into shared memory once before the K-loop, dropping runtime from 3.34 ms to 2.72 ms. The benefit compounds with K-dimension size, since each avoided global memory read is paid once per K-tile across the full loop. Shared memory caching is the key enabler for efficient hot-loop pattern lookups.

**Float2 A K-tile loads (ESMM vs. K26):** −5% **at 12.5% density.** Float2 loads raise thread utilization during A K-tile loading from 50% to 100%. The gain is modest at low density where A-load time is small relative to total runtime, but more meaningful at moderate densities where tile loads dominate compute.

**Figure 4: NCU compute times for all kernel variants at each block density. ESMM achieves the lowest time at all densities below 50%. K27 demonstrates that finer granularity without block-level skipping provides no benefit.**

## 3.4 Skip Rate Analysis

Table 3 reports the fraction of K-tiles eliminated at each skip level alongside ESMM compute time. Figure 5 visualizes the skip breakdown by level.

**Table 3: Three-level skip rates and ESMM compute time, synthetic blockwise,** 4096 × 4096**.** ***dotIdx executed*** **= fraction of inner-loop iterations that run. At 12.5% density, block and warp-level skips together eliminate the vast majority of computations, leaving only 1.6% of dotIdx iterations active.**

| Block Density | Block-skip | Warp-skip | dotIdx exec'd | ESMM compute tim |
|---|---|---|---|---|
| 100% | 0.0% | 0.0% | 100% | 9.99 m |
| 50% | 0.0% | 10.0% | 25% | 5.80 m |
| 25% | 6.0% | 59.8% | 6.2% | 3.99 m |
| 12.5% | **44.0%** | **88.1%** | **1.6%** | **2.63 m** |

The skip hierarchy is progressive: coarse-grained skipping dominates at lower density, while finer levels handle residual sparsity.

At 50% density no K-tiles are block-skippable, but warp-level skips eliminate 10% of warp-K-tile pairs and the joint condition reduces dotIdx executions to 25% of the dense baseline. At 12.5% density: **44% of K-tiles are eliminated at block level** before any memory is touched; warp-level skips handle 88.1% of the remaining warp pairs; only **1.6% of dotIdx iterations execute**.

Despite 98.4% of inner-loop iterations being skipped at 12.5% density, the speedup is 2.65× rather than the ∼60× skip rates alone would suggest. Three factors explain this gap:

(1) **Memory-latency bound**: active compute is limited by L2/DRAM latency, not arithmetic throughput — skipping FMAs does not fully translate to wall-clock savings.
(2) **Synchronization barriers**: each active K-tile pays two barriers regardless of how many warps skip.
(3) **Fixed preprocessing cost**: the ∼374 $\mu$s preprocessing establishes a performance floor independent of sparsity.

These results confirm that ESMM efficiently exploits block structure where it exists, but realized speedup is bounded by memory latency and fixed preprocessing costs rather than skip rates alone.

**Figure 5: Skip rate breakdown by level at each synthetic block density. At 12.5% density, 44% of K-tiles are eliminated at block level before any memory is accessed; only 1.6% of dotIdx iterations execute.**

## 3.5 Real-Weight Evaluation

While synthetic experiments confirm ESMM's efficiency when block structure exists, real transformer weights do not necessarily exhibit such structure. We therefore evaluate ESMM on pruned LLaMA weights to assess practical applicability. Table 4 reports results on LLaMA-7B Layer 0 weight tensors pruned with Wanda and SparseGPT, paired with a dense random activation matrix $A$. Figure 6 shows end-to-end times against cuBLAS.

ESMM does not outperform cuBLAS on any real-weight configuration. The results reveal a structural mismatch between how current pruning methods distribute zeros and what ESMM requires at $32 \times 8$ tile granularity.

**Why standard pruning does not produce block sparsity.** Three compounding effects explain the near-zero block-skip rates below 99% element sparsity:

(1) **Group-level constraint**: Wanda and SparseGPT preserve at least one nonzero per group of 8 or 16 elements, making it impossible to zero an entire $BK = 8$ K-tile at ≤50% element sparsity.
(2) **K-tile coverage**: for a $32 \times 8$ tile to be skippable, every group within every row must be fully zeroed — a condition requiring extreme (≥99%) element sparsity.
(3) **Wide MLP projections**: with $K = 11008$, every $BK = 8$ slice almost certainly contains a surviving element across the full column span, explaining why MLP block-skip rates reach only 0.2% even at 99% element sparsity.

**Table 4: ESMM on real LLaMA-7B weights. Element Sparsity (Elem Sp.) is the fraction of zero elements; Block-skip is the fraction of $32 \times 8$ K-tiles entirely zero. ESMM times include ∼375 $\mu$s preprocessing. ESMM does not exceed cuBLAS at any configuration.**

| Config | Shape | Elem Sp. | Blk-skip | ESMM | Speedup |
|---|---|---|---|---|---|
| Wanda grp8, 50%, attn | $4096^2$ | 50% | 0.0% | 10.1 ms | 0.71× |
| Wanda grp8, 99%, attn | $4096^2$ | 99% | 22.3% | 8.6 ms | **0.84**× |
| Wanda grp8, 50%, mlp | $4096 \times 11008$ | 50% | 0.0% | 26.4 ms | 0.72× |
| Wanda grp8, 99%, mlp | $4096 \times 11008$ | 99% | 0.2% | 25.5 ms | 0.74× |
| SparseGPT grp16, 50%, attn | $4096^2$ | 50% | 0.0% | 10.1 ms | 0.71× |
| SparseGPT grp16, 99%, attn | $4096^2$ | 99% | 0.1% | 7.8 ms | 0.92× |

At 50% element sparsity, block-skip rates are 0% across all configurations and ESMM runs at 0.71× cuBLAS, slower due to preprocessing overhead with no skip savings to offset it. At 99% element sparsity — extreme by practical deployed-model standards (practical sparsity levels are generally ≤95%) — Wanda-pruned attention matrices reach 22.3% block skips and 0.84× speedup, the best result observed. SparseGPT reaches only 0.1% block skips and 0.92×.

**Limitation.** ESMM's performance is contingent on block-structured sparsity at $32 \times 8$ granularity. Current post-training pruning methods do not produce this structure at practical sparsity levels. The synthetic results confirm that the kernel is correct and efficient when block structure exists; the real-weight results identify pruning methodology as the bottleneck. Future work should investigate pruning methods that align zero patterns with $32 \times 8$ K-tile boundaries. Addressing this via block-aware pruning is the key step for realizing ESMM's full potential in practical LLM inference.

# 4 Related Work

We categorize related work into four areas: static sparse GEMM, hardware-structured sparsity, dynamic activation sparsity, and pruning methods.

**Static Sparse GEMM.** cuSPARSE [8], Sputnik [4], Flash-LLM [10], and SpInfer [2] achieve strong results for weight sparsity by converting matrices into sparse formats offline. SpInfer explicitly excludes dynamic activation sparsity from its scope. All assume sparsity structure is known before kernel launch, making them inapplicable to activations that change every forward pass.

**Hardware-Structured Sparsity.** NVIDIA 2:4 sparsity [7] enables Tensor Core acceleration but requires retraining to conform to a fixed sparsity ratio and provides no mechanism for exploiting runtime activation sparsity.

**Figure 6: ESMM end-to-end time vs. cuBLAS on real LLaMA-7B weight tensors. ESMM does not exceed cuBLAS at any configuration; the gap narrows only at extreme (99%) element sparsity on square attention matrices.**

**Dynamic Activation Sparsity.** Deja Vu [6] predicts inactive neurons via a learned module; PIT [1] permutes matrices into dense subproblems but requires model-aware retraining. Both require modifications beyond standard deployment. ESMM, in contrast, detects sparsity directly from activation values at runtime with no predictor or retraining.

**Pruning Methods.** SparseGPT [3] and Wanda [9] achieve high element-wise sparsity post-training but preserve at least one nonzero per group, structurally preventing tile-level zeros below 99% sparsity, as we show in Section 3. This limitation motivates block-aware pruning methods that would maximize runtime skip opportunities, complementary to ESMM's inline detection.

## 5 Conclusion

We presented ESMM, a CUDA kernel family that exploits joint block-structured sparsity in both activation and weight matrices at runtime, enabling K-tile skipping without format conversion, model modification, or retraining. The key insight — that the bitwise AND of per-matrix sparsity patterns enables warp-uniform skip decisions across three hierarchical levels — allows ESMM to eliminate up to 98.4% of inner-loop iterations at 12.5% block density, yielding 2.65× compute speedup and 2.32× end-to-end speedup over cuBLAS on NVIDIA A10G.

Evaluation on real LLaMA-7B weights shows that standard group-structured pruning methods do not produce block sparsity at the $32 \times 8$ granularity ESMM requires, limiting practical gains to extreme (≥99%) element sparsity levels. This finding reframes the open problem: the kernel infrastructure for exploiting runtime sparsity is already in place, but current pruning methods do not expose sparsity at the granularity required for efficient execution. We therefore identify block-aware pruning — methods that optimize directly for tile-level rather than element-level sparsity — as a critical direction for realizing the full potential of sparsity-aware kernels in practical LLM inference.

## References


[1] Anonymous. 2023. Exploiting the Intrinsic Sparsity of LLMs with PIT. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP).*

[2] Anonymous. 2025. SpInfer: Leveraging Large-Weight Sparsity for Efficient LLM Inference on GPUs. In *Proceedings of the European Conference on Computer Systems (EuroSys).*

[3] Elias Frantar and Dan Alistarh. 2023. SparseGPT: Massive Language Models Can Be Accurately Pruned in One Shot. In *Proceedings of the International Conference on Machine Learning (ICML).*

[4] Trevor Gale, Matei Zaharia, Cliff Young, and Erich Elsen. 2020. Sparse GPU Kernels for Deep Learning. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis (SC).*

[5] Zonglin Li, Chong You, Srinadh Bhojanapalli, Daliang Li, Ankit Singh Rawat, Sashank J Reddi, Ke Ye, Felix Chern, Felix Yu, Ruiqi Guo, et al. 2023. Lazy Neuron Phenomenon: On Emergence of Activation Sparsity in Transformers. In *Proceedings of the International Conference on Learning Representations (ICLR).*

[6] Zichang Liu, Jue Wang, Tri Dao, Tianyi Zhou, Binhang Yuan, Zhao Song, Anshumali Shrivastava, Ce Zhang, Yuandong Tian, Christopher Re, and Beidi Chen. 2023. Deja Vu: Contextual Sparsity for Efficient LLMs at Inference Time. In *Proceedings of the International Conference on Machine Learning (ICML).*

[7] NVIDIA Corporation. 2021. Accelerating Inference with Sparsity Using the NVIDIA Ampere Architecture and NVIDIA TensorRT. https://developer.nvidia.com/blog/accelerating-inference-with-sparsity-using-ampere-and-tensorrt.

[8] NVIDIA Corporation. 2024. cuSPARSE Library. https://docs.nvidia.com/cuda/cusparse.

[9] Mingjie Sun, Zhuang Liu, Anna Bair, and J. Zico Kolter. 2024. A Simple and Effective Pruning Approach for Large Language Models. In *Proceedings of the International Conference on Learning Representations (ICLR).*

[10] Haojun Xia, Zhen Zheng, Yuchao Li, Donglin Zhuang, Zhongzhu Zhou, Xiafei Qiu, Yong Li, Wei Lin, and Shuaiwen Leon Song. 2023. Flash-LLM: Enabling Cost-Effective and Highly-Efficient Large Generative Model Inference with Unstructured Sparsity. In *Proceedings of the VLDB Endowment*, Vol. 17.